

Smart Parking Lot

Manual de implementación — Interfaz gráfica WinUI 3 nativa Windows 11

Versión 1.0 · .NET 10 · Windows App SDK 1.6 · Mayo 2026

1. Resumen ejecutivo

Se desarrolló una interfaz gráfica nativa para el sistema Smart Parking Lot, replicando el 100% de las funciones del menú de consola existente. La GUI fue construida sobre WinUI 3 con el Windows App SDK 1.6, lo cual garantiza una apariencia y comportamiento idénticos a las aplicaciones nativas de Windows 11 (Mica, modo oscuro automático, color de acento del sistema, controles Fluent).

La nueva aplicación coexiste con la CLI original: ambas comparten las capas Core, Application, Hardware y Persistence sin modificaciones. Solo se agregó un nuevo proyecto SmartParkingLot.Gui que reusa toda la lógica de dominio.

2. Decisiones de diseño

2.1 Elección del framework

El mockup hi-fi (Smart Parking Hi-Fi.html) utiliza el sistema de diseño WinUI 3 / Fluent Design con Mica, acrílico, segmented controls y la paleta de Windows 11. Para llevarlo fielmente a una aplicación de escritorio se evaluaron tres opciones:

Framework	Fidelidad visual	Veredicto
WinForms	Baja — sin Mica, requiere custom paint	Descartado
WPF + WPF-UI	Alta — aproxima Fluent con librería externa	Aceptable
WinUI 3 (Windows App SDK)	Completa — Mica/acrílico nativos	ELEGIDO

Se eligió WinUI 3 porque ofrece el efecto Mica real, el sistema de temas claro/oscuro automático del SO, el color de acento del usuario y los controles Fluent (NavigationView, AutoSuggestBox, ToggleSwitch) sin necesidad de paquetes adicionales.

2.2 Arquitectura — coexistencia con la CLI

En lugar de reemplazar el ParkingLotApp.RunAsync() de la CLI, se extrajo todo el cableado de dependencias a una clase ParkingServices.BootstrapAsync() que devuelve objetos listos para usar

(ParkingLot, GateController, IParkingRepository, IEventPublisher, IArduinoReader, etc.). La CLI sigue intacta; la GUI consume los mismos servicios.

Esto produce tres beneficios:

- **Reutilización total:** cero duplicación de lógica de negocio. Cualquier corrección en Core/Application impacta a ambas interfaces.
- **Pruebas paralelas:** se puede correr la CLI y la GUI en paralelo apuntando a la misma BD SQLite para verificar la coherencia.
- **Tolerancia a Arduino ausente:** si el puerto COM no existe, la GUI muestra "desconectado" en el status bar y sigue funcionando offline; la CLI tenía el mismo comportamiento.

3. Estructura del nuevo proyecto

Se creó la carpeta src/Gui/ con los siguientes archivos clave (todos compilados como parte del proyecto SmartParkingLot.Gui.csproj):

Archivo	Responsabilidad
SmartParkingLot.Gui.csproj	Proyecto WinUI 3 unpackaged, self-contained, win-x64
app.manifest	DPI awareness + supported OS para Windows 11
App.xaml / App.xaml.cs	Bootstrap de la app, splash, captura de excepciones
MainWindow.xaml / .cs	Shell con title bar, NavigationView, status bar y Mica
Styles/Theme.xaml	Paleta semántica (Success, Warning, Danger, Tx1–Tx3)
Bootstrap/ParkingServices.cs	Cableado headless del dominio (mirror de ParkingLotApp)
Bootstrap/HardwareConfig.cs	Lectura de hardware.json
Bootstrap/GuiLogger.cs	Logger en memoria con ring-buffer para el log en vivo
Bootstrap/GuiConstants.cs	Constantes (DEFAULT_LOT_ID, gate IDs, pines...)
Controls/Badge.cs	Factory de chips Fluent (Success/Warning/Danger/Accent/Neutral)
Pages/DashboardPage.*	Tiles, ocupación por zona, alertas, resumen gates
Pages/MapPage.*	Mapa interactivo de spots + control de gates AUTO/MANUAL
Pages/LogPage.*	Búsqueda en historial (placas, sensores, dispositivos)
Pages/AdminPage.*	Lista completa de spots desde la BD
Pages/HardwarePage.*	Conexión Arduino, log en vivo, simulación de sensores
Pages/SplashPage.cs	Splash + página de Configuración
hardware.json	Mapeo de sensores → spots y gates → pines

4. Cobertura funcional — CLI ↔ GUI

Las 11 funciones del menú de consola están todas accesibles desde la GUI. La tabla siguiente muestra el mapeo función → ubicación en la interfaz gráfica.

Función del menú CLI	Ubicación en la GUI
Solicitar entrada de vehículo	Mapa de Spots → botón "Solicitar entrada"
Solicitar salida de vehículo	Mapa de Spots → botón "Solicitar salida"
Actualizar estado de espacio (sensor manual)	Mapa de Spots → click directo sobre el spot
Ver estado del parqueadero	Dashboard (tiles + ocupación por zona)
Ver historial de un vehículo	Historial → tipo "Solicitudes (placa)"
Ver lecturas de un sensor	Historial → tipo "Lecturas de sensor"
Ver acciones de un dispositivo	Historial → tipo "Acciones de dispositivo"
Monitoreo en tiempo real (Arduino)	Hardware → panel "Log en vivo"
Ver estado de espacios (BD)	Gestión de Spots (tabla completa con filtros)
Ver logs recientes	Hardware → "Archivo de log de hoy"
Simular sensor de puerta (IR)	Mapa o Hardware → "Simular IR entrada/salida"

5. Recorrido por las páginas

5.1 Dashboard

Pantalla principal con cuatro indicadores:

- Spots ocupados / Disponibles / Solicitudes hoy / % Ocupación
- Barras de progreso por zona con color semántico (verde < 70% < amarillo < 90% < rojo)
- Lista de alertas recientes leídas del GuiLogger (entradas Warning/Error)
- Tarjetas resumen de cada gate con su estado y tipo (ENTRY/EXIT)

5.2 Mapa de Spots

Núcleo interactivo de la aplicación. Renderiza los spots agrupados por zona usando `VariableSizeWrapGrid`. Cada celda es clicable: al hacer click se publica un `SensorReadingReceived` en el bus de eventos, exactamente igual que en la CLI.

En el panel derecho se controlan los gates:

- Cambio de modo AUTOMATIC ↔ MANUAL por gate
- Botón Abrir/Cerrar manual cuando está en modo MANUAL
- Badge con el estado actual (Abierta/Cerrada)

Los dos botones de la command bar (Solicitar entrada / Solicitar salida) abren un `ContentDialog` que pide la placa, crean un `EntryRequest` o `ExitRequest` y lo despachan al `GateController`. La respuesta (concedido/denegado) se muestra en otro `ContentDialog`.

5.3 Historial

Consulta unificada a `IParkingRepository` con tres modos según el `ComboBox`:

- Solicitudes por placa: `GetRequestHistoryAsync`
- Lecturas de sensor: `GetSensorReadingsAsync`
- Acciones de dispositivo: `GetDeviceActionsAsync`

Los resultados se renderizan en filas con timestamp, badge de tipo, detalle y referencia (RequestId / SensorId / ActionId).

5.4 Gestión de Spots

Lista todos los spots de la base de datos (no del lote en memoria) usando `GetSpotsByLotIdAsync`. Soporta búsqueda por texto sobre ID/tipo/dirección y filtro por tipo (Estándar/PMR/Moto). Footer con el conteo "X de Y spots".

5.5 Hardware / Arduino

Centro de control y diagnóstico del bridge serial:

- Estado de conexión (Conectado/Desconectado) con badge animado
- Botón Conectar/Desconectar que llama `Bridge.StartListening / StopListening`
- Escaneo de puertos COM disponibles vía `SerialPort.GetPortNames()`
- Log en vivo suscrito al `GuiLogger`, con toggle de auto-scroll
- Publicación manual de eventos de sensor (cualquier sensor del sistema)
- Simulación de IR para entrada/salida (mismo evento que la CLI)
- Tail del archivo de log físico del día (últimas 80 líneas)

6. Detalles técnicos relevantes

6.1 Mica y title bar custom

La ventana usa Mica como backdrop nativo cuando el SO lo soporta (Windows 11), con fallback a acrílico. El title bar es un Grid custom declarado en XAML y registrado con `ExtendsContentIntoTitleBar = true + SetTitleBar(AppTitleBar)`, lo que produce el área de arrastre nativa sin redibujar nada.

```
private void TryApplyBackdrop()
{
    if (MicaController.IsSupported())
        SystemBackdrop = new MicaBackdrop { Kind = MicaKind.Base };
    else if (DesktopAcrylicController.IsSupported())
        SystemBackdrop = new DesktopAcrylicBackdrop();
}
```

6.2 Paleta semántica con ThemeDictionaries

En Styles/Theme.xaml se definen dos diccionarios — Light y Dark — con la misma lista de claves (SuccessBrush, WarningBrush, DangerBrush, Tx1Brush a Tx3Brush, Layer1Brush, Layer2Brush, etc.). WinUI cambia automáticamente el set activo según el tema del SO.

6.3 Self-contained deployment

El .csproj usa `WindowsAppSDKSelfContained=true + SelfContained=true` para que el ejecutable final incluya tanto el runtime de .NET 10 como el de Windows App SDK 1.6. Esto evita el error clásico "This application requires the Windows App Runtime 1.6". El binario pesa ~150 MB pero corre en cualquier Windows 10/11 x64 sin instalar nada.

6.4 Captura de excepciones tempranas

`App.OnLaunched` envuelve la construcción de `MainWindow` y el bootstrap en try/catch separados, y `App.UnhandledException` tiene un fallback que muestra un `MessageBox Win32` cuando todavía no hay ventana WinUI donde mostrar el error. Cada paso del arranque se registra en `%TEMP%\SmartParkingLot.Gui-crash.log` para diagnóstico.

7. Cómo compilar y ejecutar

Opción A — Desde la línea de comandos

```
cd C:\Dev\smart-parking-lot\src\Gui
dotnet run -c Debug -r win-x64
```

Opción B — Ejecutable directo

Después de compilar una vez, basta doble click sobre:

```
C:\Dev\smart-parking-lot\src\Gui\bin\Debug\
net10.0-windows10.0.19041.0\win-x64\SmartParkingLot.Gui.exe
```

Opción C — Visual Studio

Abrir `smart-parking-lot.sln`, marcar `SmartParkingLot.Gui` como Startup Project, cambiar plataforma de Any CPU a x64 y pulsar F5.

8. Notas finales

- La GUI no rompe la CLI: ambos proyectos compilan y se ejecutan independientemente.
- `src/Gui/hardware.json` tiene un mapeo expandido (9 spots) para que el mapa luzca; en producción debe alinearse con el cableado real del Arduino.
- Los HTMLs del mockup se conservaron en `src/Gui/` como referencia visual; no se incluyen en la compilación.
- Si la ventana no aparece al ejecutar, revisar `%TEMP%\SmartParkingLot.Gui-crash.log` donde se registra cada paso del arranque y cualquier excepción.